0. Orientation: How Networks Work and How to Practice

00. Orientation: How Networks Work and How to Practice

Phase: Phase 0 — Orientation
Estimated time: 1 day
Slug: `00-orientation`

Learning objectives

- Explain clients, servers, packets, and protocols at a beginner level
- Set up a safe practice lab (VM + localhost tools)
- Adopt attempt → struggle → solution → rewrite
- Know which tools you will use daily

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

It is 02:14 and a product page is 'down'. Stakeholders ask if it is DNS, the load balancer, TLS, or the app. Without a mental model of clients, packets, ports, and protocols, you jump between dashboards. Orientation trains a safe lab habit and a daily toolkit so every later lesson maps to something you can type under pressure.

Core concepts

- **Host**: machine with interfaces; **packet/frame**: chunk with headers; **protocol**: agreed rules.
- **Port**: application endpoint on a host (TCP/UDP). Same IP can host many services via different ports.
- **Client/server**: initiator vs listener — but roles flip (e.g. callbacks, peer protocols).
- Practice loop: attempt → struggle → solution → rewrite. Typing beats highlighting.
- Lab stack: laptop terminal + Linux VM (`ip`, `ss`, `nft`) + optional second VM for path tests.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
ping -c 3 1.1.1.1
dig example.com +short
curl -sS -o /dev/null -w '%{http_code} %{time_total}\n' https://example.com
```

Sample output:

```
64 bytes from 1.1.1.1: icmp_seq=1 ttl=57 time=12.1 ms
...
93.184.216.34
200 0.182
```

Interpretation: ICMP reachability, DNS A answer, and HTTP status/latency are three different layers of evidence. Collect all three before blaming 'the network'.

Example 2

```
# Linux
ss -tulpn | head
ip -br addr
# macOS analogues
# netstat -anv | head; ifconfig en0
```

Sample output:

```
udp UNCONN 0 0 0.0.0.0:53 ...
tcp LISTEN 0 128 0.0.0.0:22 ...
eth0 UP 192.168.1.20/24
```

Interpretation: `ss` shows listeners (who accepts connections). `ip -br addr` shows L3 addresses on interfaces. Knowing both prevents 'port open' vs 'wrong IP' confusion.

Example 3

```
traceroute -n 1.1.1.1 2>/dev/null | head -5 || traceroute 1.1.1.1 | head -5
```

Sample output:

```
1 192.168.1.1 1.2 ms
2 10.0.0.1 8.4 ms
3 1.1.1.1 12.0 ms
```

Interpretation: Path hops are clues, not proof of the fault. Firewalls may hide hops (`* * *`). Still useful to see whether you leave the LAN.

Mini-lab

1. Create a `lab-notes.md` and paste outputs of ping, dig, curl write-out, and ss/netstat.
2. Start a tiny listener (`python3 -m http.server 8080`) and curl `http://127.0.0.1:8080/` — prove client/server locally.
3. List five tools you will use daily and one sentence each on when.
4. Tomorrow: rewrite the greetings/tooling exercise from memory before reopening solutions.

Failure modes

Symptom	Likely cause	Next check
Only reading PDFs	No motor memory	Type every example
Practicing on prod	Blast radius / policy risk	Use VMs and approved lab nets
Jumping to cloud GUIs	Skips fundamentals	Finish host toolkit first
`curl` fails, blame DNS immediately	Wrong layer guess	Check dig and ping separately

Study tip for Orientation: How Networks Work and How to Practice

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Only reading without typing commands
- Practicing against production systems without approval
- Skipping fundamentals and jumping to cloud GUIs

Exercises

Work in `exercises/00-orientation/`. Read that folder's `README.md` before coding.

```
cd exercises/00-orientation
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Implement three helpers: `greet` (friendly hello), `daily_tool_count_min` (minimum toolkit size taught: 5), and `study_loop` (the four-step practice phrase). Comments above each function should say *when* you use the idea on-call.

Study loop

- **A (Drill):** make tests pass

- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/00-orientation/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. What is a port?
2. Name five daily networking commands
3. Why use a lab VM?

Further reading

- https://www.rfc-editor.org/rfc/rfc1122